

1. FRONTEND FUNDAMENTALS

Before writing our first line of code, let's understand how the web works and how the browser turns code into a beautiful webpage.

1.1 What is Frontend?

Frontend (Client-side) is everything that the user sees and interacts with in the browser.

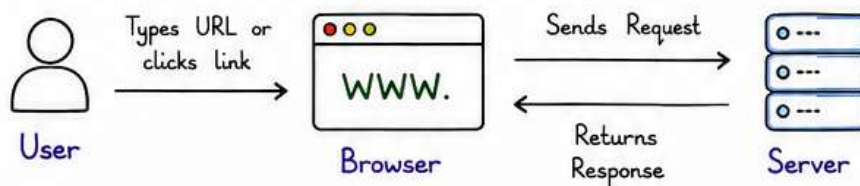
Examples:

HTML, CSS, JavaScript, Images, Fonts, UI Libraries, Frameworks (React, Angular, Vue)

1.2 Client vs Server

Client (Frontend)	Server (Backend)
- Runs in the browser - Renders UI - Handles user interactions - Sends requests to server - Technologies: HTML, CSS, JS, React, etc.	- Runs on the server - Holds business logic - Handles database - Sends responses to client - Technologies: Node.js, Java, Python, etc.

1.3 How Browsers Work? (High Level)



- Browser sends HTTP request to the server.
- Server processes the request and sends back HTML, CSS, JS, Images, etc.
- Browser downloads and renders the content.

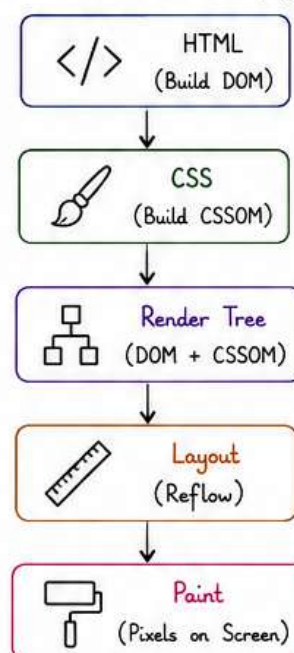
1.4 Browser Rendering Pipeline

The browser converts code into pixels on the screen in this order:

- HTML Parsing (DOM Tree)**
Browser parses HTML and creates the DOM (Document Object Model).
 - CSS Parsing (CSSOM Tree)**
Browser parses CSS and creates the CSSOM (CSS Object Model).
 - Render Tree**
DOM + CSSOM combine to form the Render Tree (what needs to be shown).
 - Layout (Reflow)**
Calculates the position and size of each element on the screen.
 - Painting**
Browser paints the pixels on the screen.
- * Reflow (Layout)**
When size or position of elements change, browser recalculates the layout.
- * Repaint**
When only style (color, background etc.) changes, browser repaints without redoing the layout.

1.5 Critical Rendering Path

This is the optimized path the browser follows to render the page as fast as possible.



★ Anything that blocks this path will delay the first render.

Examples:

- Render blocking CSS/JS
- Large CSS
- Synchronous JS
- Unoptimized Images/Fonts



TIP:

Understand the basics deeply. Frontend is not just about frameworks, it's about how the web works! ❤️

IG: abhi_techhub



2. HTML INTERVIEW ESSENTIALS

HTML is the skeleton of any webpage. Let's cover the most important concepts.

2.1 What is HTML?

HTML (HyperText Markup Language) is used to structure content on the web.

Example:

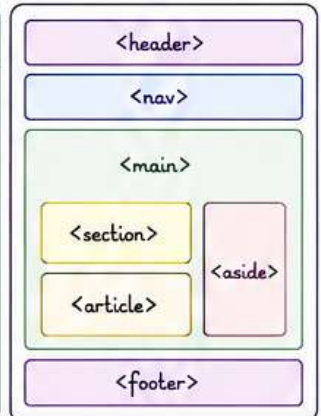
```
<h1>Hello World</h1>
<p>This is a paragraph.</p>
<a href="https://google.com">Google</a>
```

Browsers read HTML and convert it into visible content.

2.2 Semantic HTML

Use semantic elements for better SEO, accessibility and code readability.

Element	Used For
<code><header></code>	Represents the top part (logo, heading, nav)
<code><nav></code>	Navigation links
<code><main></code>	Main content of the page (only one per page)
<code><section></code>	Thematic grouping of content
<code><article></code>	Self-contained content (blog post, news, etc.)
<code><aside></code>	Sidebar or related content
<code><footer></code>	Bottom part (copyright, links, info)



2.3 HTML Forms

Important form elements:

- `<form>`: Wraps form elements
- `<input>`: Text, email, checkbox, radio, etc.
- `<label>`: Label for input (accessibility)
- `<textarea>`: Multi-line input
- `<select>`: Dropdown
- `<button>`: Clickable button

```
<form action="/submit" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" />
  <br/><br/>
  <label for="age">Age:</label>
  <input type="number" id="age" name="age" />
  <br/><br/>
  <button type="submit">Submit</button>
</form>
```

2.4 script: async vs defer

- async**: Script downloads in parallel and executes as soon as it is downloaded. Does not block HTML parsing.
- defer**: Script downloads in parallel but executes after HTML is fully parsed, in order.

```
<script src="a.js" async></script>
<script src="b.js" defer></script>
<script src="c.js"></script>
↑
(Block HTML parsing)
```

2.5 Important <meta> Tags

- `<meta charset="UTF-8">`
Sets character encoding
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">` (Responsive)
- `<meta name="description" content="...">`
For SEO (search engines)
- `<meta name="keywords" content="html, css, js">`
(Not very important now)

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,
    initial-scale=1.0" />
  <meta name="description" content="Frontend
    Interview Handbook" />
  <title>My Page</title>
</head>
```

2.6 Accessibility (a11y)

Make your HTML accessible for all users.

- ✓ Use semantic HTML.
- ✓ Always use `<label>` with form controls.
- ✓ Add `alt` text to images.
- ✓ Proper heading order (h1 → h2 → h3).
- ✓ Use ARIA roles when needed.

```

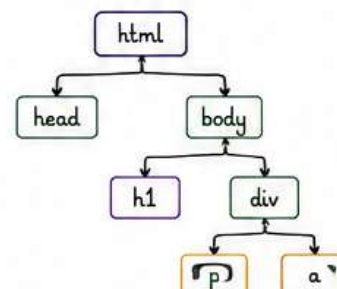
<button aria-label="Close dialog">X</button>
```

2.7 SEO Basics

- ✓ Use semantic HTML.
- ✓ Add meaningful `<title>`.
- ✓ Add meta description.
- ✓ Use heading tags properly.
- ✓ Add alt text to images.
- ✓ Make page mobile-friendly.
- ✓ Use clean URLs.

2.8 DOM (Document Object Model)

HTML is converted into a tree-like structure.



★ JavaScript uses DOM to access and update HTML content.



TIP:

Write clean, semantic and accessible HTML.

It helps users, developers and search engines! ❤️

IG: abhi_techhub

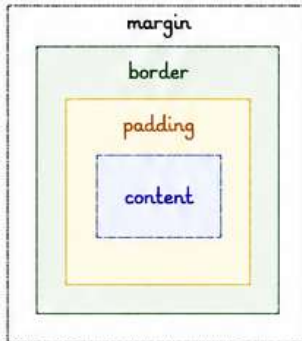


3. CSS INTERVIEW ESSENTIALS

PAGE 3

CSS is all about styling, layout and making the UI beautiful and responsive.

3.1 The Box Model



- **Content:** The actual content (width/height)
- **Padding:** Space inside the border
- **Border:** Surrounds the padding
- **Margin:** Space outside the border (separates from other elements)

Total Width = margin + border + padding + content width
Total Height = margin + border + padding + content height

3.2 Specificity & Cascade

Selector Type	Specificity Value
!important	∞ (highest)
Inline Styles	1-0-0-0
ID Selector (#id)	0-1-0-0
Class (.class), Attribute ([type]), Pseudo-class (:hover)	0-0-1-0
Element (div), Pseudo-element (::before)	0-0-0-1
Universal (*)	0-0-0-0

Cascade Order (when specificity is same):

Importance > Specificity > Source Order (last one wins)

3.3 Positioning

Position	Description
static	Default position. Follows normal document flow.
relative	Relative to its normal position. Offset using top, right, bottom, left .
absolute	Removed from normal flow. Positioned relative to nearest positioned ancestor.
fixed	Removed from flow. Positioned relative to viewport. Stays in place on scroll.
sticky	Acts like relative until a scroll threshold, then sticks like fixed.

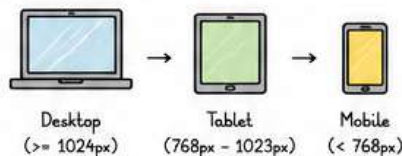
3.4 Flexbox vs Grid

Flexbox (1D Layout)	CSS Grid (2D Layout)
• Works in one direction (row OR column). • Best for aligning items in a line. • Use for navbars, menus, components in a row etc. Example:	• Works in two dimensions (rows AND columns). • Best for complex layouts. • Use for dashboards, layouts with rows & columns. Example:

3.5 Units in CSS

Unit	Description
px	Absolute unit. Fixed size.
%	Relative to parent element.
em	Relative to font-size of parent.
rem	Relative to font-size of root (<html>).
vh / vw	1% of viewport height / width.
ch	Width of '0' (used for text).

3.6 Media Queries (Responsive Design)



★ Mobile First Approach is recommended!

```
/* Example */
@media (max-width: 1023px) {
  .container { flex-direction: column; }
}

@media (max-width: 767px) {
  !.menu { display: none; }
}
```

3.7 Common CSS Properties

Display	: block, inline, inline-block, flex, grid, none
Text	: color, text-align, text-decoration, line-height
Background	: background-color, background-image, background-size
Spacing	: margin, padding
Border	: border, border-radius
Others	: width, height, overflow, box-shadow, opacity, z-index

★ Tips:

- Use rem for scalable font sizes.
- Use Flexbox for 1D layouts.
- Use Grid for 2D layouts.
- Keep CSS clean, reusable and well-structured.

3.8 CSS Methodologies

- Inline CSS (Not scalable)
 - Internal CSS (Inside <style>)
 - External CSS (Best Practice)
- Always prefer External CSS with meaningful class names.



TIP:

Master the basics. Write clean CSS.

Your future self (and teammates) will thank you! ❤️

IG: abhi_techhub



4. JAVASCRIPT CORE - PART 1

PAGE 4

Let's build strong JS fundamentals. These are the most asked in interviews.

4.1 var, let and const

Keyword	Scope	Hoisting	Re-declare	Re-assign
var	Function Scoped	Hoisted (undef. init)	Yes	Yes
let	Block Scoped	Hoisted (Temporal Dead Zone)	No	Yes
const	Block Scoped	Hoisted (Temporal Dead Zone)	No	No (must init)

- ★ Note:
- var is function scoped and can lead to bugs.
 - Prefer let and const in modern JavaScript.

4.2 Hoisting

- Hoisting is JS behavior of moving declarations to the top of their scope during the compilation phase.
- Only declarations are hoisted, not initializations.

Example:

```
// var
console.log(a); // undefined
var a = 10;
```

Output: undefined

```
// let / const
console.log(b); // ReferenceError
let b = 20;
```

Output: ReferenceError

4.3 Scope in JavaScript

- Global Scope - Available everywhere in the program.
- Function Scope - Available inside the function.
- Block Scope - Available inside {} (let, const).

Scope Chain:

JS looks for variables in the current scope, then moves outward (lexical environment).

Local Scope → Parent Scope → Global Scope

4.4 Closures

A closure is a function that remembers variables from its lexical scope even when the outer function has finished execution.

Example:

```
function outer() {
  let count = 0;
  return function inner() {
    count++;
    console.log(count);
  }
}
const fn = outer();
fn(); // 1
fn(); // 2
fn(); // 3
```

★ Use Cases:

- Data privacy
- Function factories
- Module pattern
- Callbacks

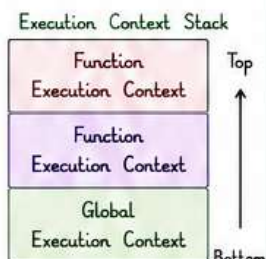
Closures are powerful and very often asked in interviews!

4.5 Execution Context

When JS code runs, an Execution Context is created.

It has two phases:

- Creation Phase (Memory Phase)
 - Allocate memory for variables and functions.
 - Initialize with undefined.
- Execution Phase (Code Phase)
 - Assign actual values.
 - Execute line by line.



4.7 Primitive vs Reference Types

Primitive (Copied by Value)	Reference (Copied by Reference)
- string - number - boolean - null - undefined - symbol - bigint - Stored directly in stack.	- object - array - function - date - regex - Stored as reference in stack, actual data in heap.

4.8 == vs ===

Operator	Checks	Type Conversion	Example	Result
==	Value equality	Yes (unsafe)	5 == '5'	true
			null == undefined	true
===	Value + Type equality	No (safe)	5 === '5'	false
			null === undefined	false

★ Always prefer === over == in interviews and real code.



TIP:

Strong JS fundamentals make complex problems easy.

Practice, solve problems and understand concepts deeply! ❤️

IG: abhi_techhub





5. JAVASCRIPT CORE - PART 2

PAGE 5

Let's go deeper into advanced JS concepts that are heavily asked in interviews.

5.1 Promises

- A Promise represents the eventual completion (or failure) of an async operation.
- States: Pending → Fulfilled or Rejected

Example:

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => {
    let ok = true;
    if (ok) resolve("Success");
    else reject("Error");
  }, 1000);
});

p.then(res => console.log(res))
  .catch(err => console.log(err))
  .finally(() => console.log("Done"));
```

Methods:

- then()
- catch()
- finally()
- all()
- race()
- allSettled()
- any()

5.2 async / await

- Built on top of Promises to write async code in a synchronous way.
- await can only be used inside async function.

Example:

```
async function getData() {
  try {
    const res = await fetch("/api/data");
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log("Error:", err);
  }
}

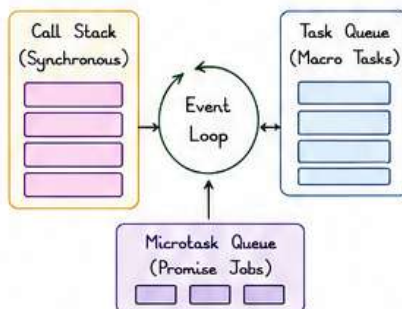
getData();
```

★ Tip:

async returns a Promise automatically.

5.3 Event Loop in JavaScript

- JS is single-threaded but async.
- Uses Event Loop to handle async operations.



- Call Stack executes the code.
- Async tasks go to Web APIs (Browser).
- On completion, callbacks go to Task Queue.
- Event Loop pushes tasks to Call Stack.
- Microtasks have higher priority than Macro tasks.

5.4 Microtask vs Macrotask

Feature	Microtask Queue	Macrotask Queue
Examples	Promises, queueMicrotask, MutationObserver	setTimeout, setInterval, setImmediate, I/O, UI Rendering
Priority	Higher (executes first)	Lower
When Runs	After current call stack is empty	After microtasks are completed
Can Starve UI?	Yes (if too many)	No

Order of Execution:

Call Stack Empty → Microtasks → Macrotask → Render (Browser) → Repeat

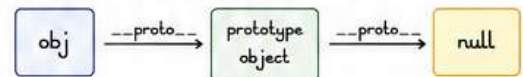
5.5 this Keyword

- this depends on how a function is called, not where it is defined.
- Rules (precedence):
 - new binding > 2. explicit binding > 3. implicit binding > 4. default binding

1. new binding	2. explicit binding	3. implicit binding	4. default binding
When used with new keyword. <pre>function Person(){ this.name = "Abhi"; } const p = new Person(); console.log(p.name); // Abhi</pre>	call(), apply(), bind() <pre>let obj = {name: "Abhi"; function show(){ console.log(this.name); } } show.call(obj); // Abhi</pre>	When called on an object. <pre>const obj = {name: "Abhi", show(){ console.log(this.name); } } obj.show(); // Abhi</pre>	In strict mode → undefined In non-strict → window <pre>function show(){ console.log(this); } show(); // undefined (strict)</pre>

5.6 Prototypes in JavaScript

- Every object has an internal link to another object called its prototype.
- JS uses prototype chain for inheritance.



Example:

```
const person = { name: "Abhi";
console.log(person.__proto__);

const arr = [];
console.log(arr.__proto__ === Array.prototype); // true
```

5.7 Common Output Questions

What will be the output?

1.

```
console.log(1);
setTimeout(() =>
  console.log(2), 0);
Promise.resolve()
  .then(() => console.log(3));
console.log(4);
```

Output: 1 4 3 2

2.

```
setTimeout(() =>
  console.log('A'), 0);
Promise.resolve()
  .then(() => console.log('B'))
  .then(() => console.log('C'));
console.log('D');
```

Output: D B C A

3.

```
for (var i = 0; i < 3; i++) {
  setTimeout(() =>
    console.log(i), 0);
}
```

Output: 3 3 3

4.

```
for (let i = 0; i < 3; i++) {
  setTimeout(() =>
    console.log(i), 0);
}
```

Output: 0 1 2

★ Remember:

- JS is single-threaded
- Event Loop is the hero
- Understand execution order deeply
- Practice more output based questions!



TIP:

- ✓ Draw diagrams while learning.
- ✓ Write code by hand.
- ✓ Solve output questions daily.

Interviews test logic + clarity.
Stay calm, think step by step
and explain your thought process.



IG: abhi_techhub



6. REACT.JS INTERVIEW ESSENTIALS

PAGE 6

React is a JavaScript library for building user interfaces using components.

6.1 What is React?

- Library for building UI.
- Component based.
- Declarative and efficient.
- Uses Virtual DOM.
- One-way data binding.
- Developed & maintained by Facebook.



6.2 Components

Functional Component

```
function Welcome(props) {
  return (
    <div className="box">
      Hello {props.name} 🐼
    </div>
  );
}
```

Class Component

```
class Welcome extends React.Component {
  render() {
    return (
      <div className="box">
        Hello {this.props.name} 🐼
      </div>
    );
  }
}
```

6.3 JSX (JavaScript XML)

- JSX allows us to write HTML inside JavaScript.
- Not mandatory, but makes code readable.
- JSX is not HTML inside JS, it gets transpiled to JS (React.createElement).

JSX:

```
const element = <h1 className="title">Hello React</h1>;
```

Transpiled JS:

```
const element = React.createElement(
  "h1",
  { className: "title" },
  "Hello React"
);
```

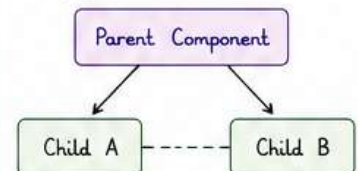
This is what browser understands.

6.4 Props vs State

Props	State
Passed from parent to child.	Managed within the component.
Read-only (immutable).	Can be updated (mutable).
Used to pass data and callbacks.	Used to manage internal data.
Changes do not affect parent.	Changes trigger re-render.

6.5 One-way Data Flow

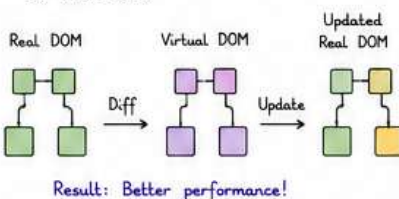
Data flows from Parent → Child via props.



Child cannot directly modify parent's data. It can only send events back via callbacks.

6.6 Virtual DOM

- Lightweight copy of real DOM.
- React updates Virtual DOM first.
- Then compares (diffing algorithm) and updates only the changed parts in Real DOM.



6.7 React Hooks (Most Used)

useState	Add state to functional components.
useEffect	Perform side effects (API calls, DOM updates, subscriptions).
useContext	Access context values.
useRef	Access/modify DOM elements or persist values.
useMemo	Memoize expensive calculations.
useCallback	Memoize functions (useful for children props).

6.8 useState Example

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

Initial value is 0

Updates state and re-renders component

6.9 useEffect Example

```
import { useEffect, useState } from "react";

function Example() {
  const [data, setData] = useState([]);

  useEffect(() => {
    fetch("https://api.example.com/data")
      .then(res => res.json())
      .then(json => setData(json));
  }, []); // Empty array -> runs only once

  return <div>{data.length} items</div>;
}
```

Runs after component mounts

Dependency array

6.10 Keys in Lists

Keys help React identify which items changed, added or removed.

Good:

```
{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

Bad (using index as key):

```
{items.map((item, index) => (
  <li key={index}>{item.name}</li>
))}
```

★ Use stable, unique IDs as keys.

6.11 Component Lifecycle (Class)

Mounting
1. constructor()
2. componentDidMount()
Updating
1. shouldComponentUpdate()
2. render()
3. componentDidUpdate()
Unmounting
1. componentWillUnmount()

Component is added to DOM

Props/State change

Component is removed from DOM



TIP:

- ✓ Build small projects.
- ✓ Understand concepts deeply.
- ✓ Practice by building and explaining.

- ✓ Read React docs.
- ✓ Keep components small & reusable.
- ✓ Write clean & maintainable code.

IG: abhi_techhub



7. REACT PERFORMANCE OPTIMIZATION

Write clean code first, then make it fast. Optimize only when needed!

7.1 Why Performance Matters?

- Better user experience (speed = happy users 😊)
- Lower bounce rate
- Saves battery & network
- Handles large scale applications efficiently
- Good Core Web Vitals (LCP, INP, CLS)
- Reduces unnecessary re-renders



Remember:

Measure → Identify → Optimize → Validate

7.2 What Causes Re-renders?

- State update
- Props change
- Parent re-render
- New reference (objects, arrays, functions) on every render
- Context value change
- Inline functions in JSX

Tip:

React re-renders a component when its props or state change.

7.3 React.memo

Prevents re-rendering if props are same.

```
const MyComponent = React.memo(function MyComponent({ name }) {
  return <div>{name}</div>;
});
```

Custom comparison:

```
const MyComponent = React.memo(Component,
  (prevProps, nextProps) => {
    return prevProps.name === nextProps.name;
  });
```

7.4 useMemo (Memoization for Values)

Memoizes expensive calculations and returns the cached result until dependencies change.

```
const expensiveValue = useMemo(() => {
  return heavyCalculation(data);
}, [data]);
```

Example:

```
const total = useMemo(() => {
  items.reduce((sum, item) =>
    sum + item.price, 0),
  [items]
);
```

★ Use when:

- Calculation is costly
- Dependency changes less frequently

7.5 useCallback (Memoization for Functions)

Returns a memoized callback function that maintains reference between renders.

```
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);
```

★ Use when:

- Passing callback to child components
- Preventing unnecessary re-renders in React.memo children

7.6 Avoid Inline Functions

✗ Bad

```
<Child onClick={() => doSomething(id)} />
```

✓ Good

```
const handleClick = useCallback(() =>
  doSomething(id), [id]);
<Child onClick={handleClick} />
```

★ Inline functions create new reference on every render!

7.7 Lazy Loading & Code Splitting

Load only what is needed.

1. React.lazy + Suspense

```
const Dashboard = React.lazy(() =>
  import('./Dashboard'));
<Suspense fallback=<div>Loading...</div>>
  <Dashboard />
</Suspense>
```

2. Route based splitting (React Router)

```
const Dashboard = lazy(() =>
  import('./Dashboard'));
<Route path="/dashboard" element=<Dashboard />/>
```

★ Reduces bundle size and initial load time.

7.8 Suspense

Lets you "wait" for something before rendering.

```
<Suspense fallback=<Loader />>
  <Profile />
  <Posts />
</Suspense>
```

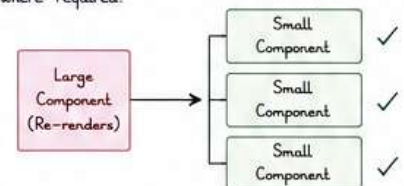
Use for:

- Code splitting
- Data fetching (with libraries)
- Better UX with fallback UI



7.9 Avoid Large Re-renders

- Keep state as local as possible.
- Lift state up only when necessary.
- Split large components into smaller ones.
- Use React.memo / useMemo / useCallback where required.



Better performance!

7.10 Unnecessary useEffect

- useEffect runs after every render by default.
- Avoid when you don't need side effects.
- Missing dependency array causes extra renders.

✗ Bad

```
useEffect(() => {
  fetchData();
}); // runs on every render
```

✓ Good

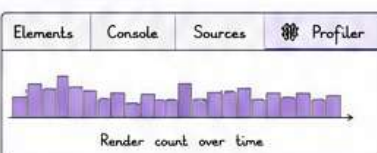
```
useEffect(() => {
  fetchData();
}, []); // runs only once
```

★ Use dependency array correctly!

7.11 Profiling (Measure First!)

Don't guess. Measure!

- Use React DevTools Profiler
- Identify slow components
- Check "Why did this render?"
- Fix bottlenecks



7.12 Performance Checklist

- ✓ Avoid unnecessary re-renders
- ✓ Use React.memo for pure components
- ✓ Memoize values with useMemo
- ✓ Memoize functions with useCallback
- ✓ Lazy load components
- ✓ Code split routes
- ✓ Use Suspense with fallback UI
- ✓ Optimize images & assets
- ✓ Profile and measure
- ✓ Keep components small & focused

Optimize only when it matters.

Premature optimization is the root of all evil. 😊

— Donald Knuth



TIP:

1. Don't optimize too early.
2. Profile, then optimize.
3. Small improvements add up!

Fast apps = Happy users 😊

Clean code + Good performance = Best combo ❤️

IG: abhi_techhub

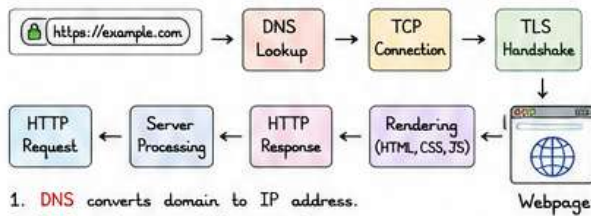


8. BROWSER + WEB FUNDAMENTALS

PAGE 8

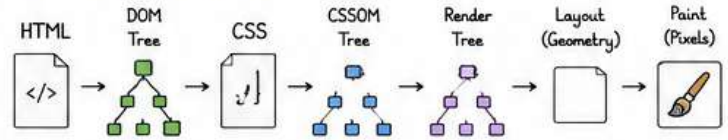
You don't need to be a browser engineer, but you should know how the web works!

8.1 From URL to Webpage



1. DNS converts domain to IP address.
2. TCP creates a reliable connection.
3. TLS handshake establishes a secure channel (HTTPS).
4. HTTP request is sent to server.
5. Server processes and sends back response.
6. Browser parses HTML, CSS → builds DOM & CSSOM.
7. JavaScript is executed → page is rendered.

8.2 Rendering Pipeline



- DOM (Document Object Model) → Structure of the page.
- CSSOM (CSS Object Model) → Styles of the page.
- Render Tree → Only visible elements from DOM + CSSOM.
- Layout → Calculates the position and size of each node.
- Paint → Converts layout into pixels on the screen.

8.3 Reflow (Layout) vs Repaint

Reflow (Layout)

- Happens when layout / geometry of elements change.
- More expensive.
- Triggers: Adding/removing elements, changing size, position, font-size, etc.

Example:

```
element.style.width = '200px';
element.style.display = 'none';
```

Repaint

- Happens when visual styles change but layout does not.
- Less expensive.
- Triggers: color, background-color, visibility, text-shadow, etc.

Example:

```
element.style.color = 'red';
element.style.background = 'blue';
```

★ Avoid unnecessary reflows & repaints for better performance!

8.5 async vs defer

Attribute	async	defer
Download	Parallel with HTML	Parallel with HTML
Execution	As soon as downloaded	After HTML parsing
Order	Not guaranteed	Order is maintained
DOM Ready	May not wait	Waits for DOMContentLoaded
Use Case	Independent scripts (analytics, ads)	Scripts that depend on DOM or other scripts

8.7 LocalStorage vs SessionStorage vs Cookies

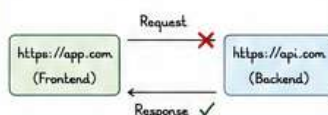
Feature	LocalStorage	SessionStorage	Cookies
Storage	Browser storage	Browser storage	Browser + Server
Lifetime	Until cleared	Until tab closed	Expires or session
Capacity	~5-10 MB	~5 MB	~4 KB
Sent to Server	X	X	✓ (with request)
Access	Via JavaScript	Via JavaScript	Via Header
Use Case	User prefs, themes	Form data, temp info	Auth, sessions

8.9 CORS (Cross-Origin Resource Sharing)

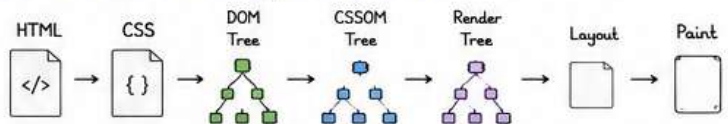
- Security feature that restricts requests from one origin to another.
- Controlled by server using headers.

Common Headers:

- Access-Control-Allow-Origin
- Access-Control-Allow-Methods
- Access-Control-Allow-Headers
- Access-Control-Allow-Credentials



8.4 Critical Rendering Path (CRP)



How to Optimize CRP?

- ✓ Minify & compress CSS/JS
- ✓ Remove unused CSS
- ✓ Inline critical CSS
- ✓ Defer non-critical JS
- ✓ Preload important resources
- ✓ Lazy load images & components
- ✓ Reduce render-blocking resources

8.6 Cookies

- Small pieces of data stored by the browser.
- Sent with every request to the same domain.
- Used for session management, auth, preferences.

Cookie Attributes:

- Expires / Max-Age → Lifetime
- Domain → Which domain can access
- Path → URL path for which cookie is valid
- Secure → Sent only over HTTPS
- HttpOnly → Not accessible via JavaScript
- SameSite → Controls cross-site requests



8.8 Caching (Browser Caching)

- Helps to reduce load time and server requests.

Types:

- Memory Cache → Fastest (temporary)
- Disk Cache → Files stored on disk
- Service Worker Cache → Offline first apps
- HTTP Cache Headers → Control caching behavior

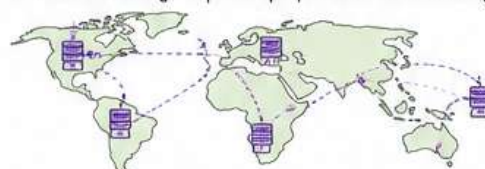
Common Headers:

- Cache-Control
- Expires
- ETag / If-None-Match
- Last-Modified / If-Modified-Since

★ Use cache wisely, but always handle cache invalidation!

8.10 CDN (Content Delivery Network)

- CDN stores static assets on servers across the world.
- Serves content from nearest location.
- Reduces latency, improves performance & availability.



Examples:

- Cloudflare
- AWS CloudFront
- Akamai
- Fastly



TIP:

- ✓ Understand how the browser works.
- ✓ Write clean, optimized & accessible code.

- ✓ Optimize performance from Day 1.
- ✓ Measure, analyze, then optimize.

IG: abhi_techhub



9. FRONTEND SECURITY + AUTHENTICATION

PAGE 9

Security is everyone's responsibility. Know the basics, avoid common mistakes.

9.1 XSS (Cross-Site Scripting)

Attacker injects malicious scripts into websites viewed by other users.

Types:

- **Stored XSS** - Script stored on server (DB).
- **Reflected XSS** - Script in URL / request.
- **DOM-based XSS** - Script runs via DOM manipulations.



Example (Bad)

```
<div>Welcome, ${userInput}</div> // userInput = <script>alert(1)</script>
```

Prevention:

- ☒ Escape user input
- ☒ Sanitize on both client & server
- ☒ Use frameworks that auto-escape (React, Angular)
- ☒ Use Content Security Policy (CSP)

9.2 CSRF (Cross-Site Request Forgery)

Attacker tricks a logged-in user to perform unwanted actions.



Prevention:

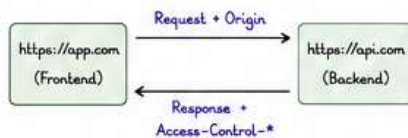
- Use CSRF Tokens (unique, secret token with each request)
- SameSite Cookies
- Check Origin / Referer header
- Re-authenticate for critical actions

9.3 CORS (Common Misconceptions)

CORS is **NOT** a security feature.

- It is a browser-enforced policy.
- It allows/blocks requests from different origins.
- If a hacker can make a request (e.g., via curl, Postman), CORS won't stop it.

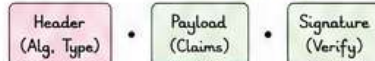
How it works:



★ Configure CORS on the server, not in the frontend.

9.4 JWT (JSON Web Token)

- Compact, URL-safe token.
- Used for authentication & authorization.
- Consists of 3 parts: Header.Payload.Signature



Storage Options:

Storage	Pros	Cons
httpOnly Cookie	Secure from XSS	CSRF risk
LocalStorage	Easy to use	Vulnerable to XSS
SessionStorage	Cleared on tab close	Vulnerable to XSS
Memory (RAM)	Most secure	Cleared on refresh

★ Best Practice: Store refresh token in httpOnly cookie, access token in memory.

9.5 Cookies

- Small key-value pairs sent with requests.
- Attributes:
 - Expires / Max-Age - lifetime
 - Domain - valid for domain
 - Path - valid for path
 - Secure - sent over HTTPS only
 - HttpOnly - not accessible via JS
 - SameSite - CSRF protection



9.6 OAuth 2.0 (Basics)

- Delegated authorization framework.
- Allows users to login using external providers (Google, GitHub, etc).
- Flow: User → Provider → App



9.7 Content Security Policy (CSP)

Helps prevent XSS and code injection attacks.

Example CSP Header:

```
Content-Security-Policy:
default-src 'self';
script-src 'self' https://cdn.com;
style-src 'self' 'unsafe-inline';
img-src 'self' data: https;
object-src 'none';
base-uri 'self';
```

Tips:

- Start with report-only mode.
- Gradually tighten the policy.
- Avoid 'unsafe-inline' whenever possible.



Always prefer Secure by Design!



9.8 Secure Token Handling

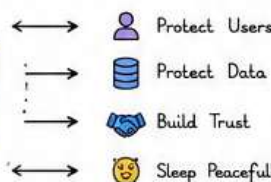
- Use short-lived access tokens.
- Rotate refresh tokens.
- Invalidate tokens on logout.
- Never expose tokens in URL.
- Validate token on every request.
- Use HTTPS everywhere.

9.9 Common Frontend Security Checklist

- ☒ Escape / sanitize user input
- ☒ Use httpOnly + Secure + SameSite cookies
- ☒ Implement CSRF protection
- ☒ Set proper CORS policy
- ☒ Use CSP headers
- ☒ Validate tokens on backend
- ☒ Avoid exposing secrets in frontend
- ☒ Keep dependencies up-to-date
- ☒ Use HTTPS for all requests

9.10 Key Takeaways

- ☒ Understand common attacks (XSS, CSRF)
- ☒ Never trust user input
- ☒ Use secure cookies & tokens wisely
- ☒ Apply least-privilege access
- ☒ Defense in depth > single protection
- ☒ Security is continuous, not one-time



A secure frontend is the first wall of defense between attackers and your users."



TIP:

- ☒ Think like an attacker, build like a defender.
- ☒ Small mistakes → Big vulnerabilities.

- ☒ Security is a feature, not a bug.
- ☒ Better to prevent than to fix later.

IG: abhi_techhub



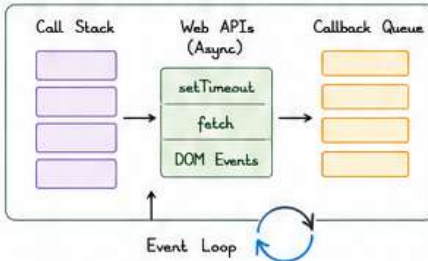
10. ADVANCED JAVASCRIPT CONCEPTS

PAGE 10

These concepts separate a good developer from a great one!

10.1 Event Loop (Deep Dive)

- JS is single-threaded.
- Uses Call Stack + Web APIs + Callback Queue.
- Event Loop pushes tasks from queue to stack when stack is empty.
- Microtasks run before Macrotasks.



★ Microtasks (Promises) > Macrotasks (setTimeout)

10.2 Closure

- A function remembers its lexical scope even when executed outside that scope.
- Used in data hiding, currying, memoization.

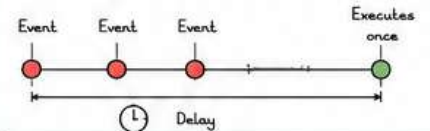
```
function outer(x) {
  let count = x;
  return function inner(y) {
    count = count + y;
    console.log(count);
  }
}

const fn = outer(5);
fn(2); // 7
fn(3); // 10 (count is remembered!)
```

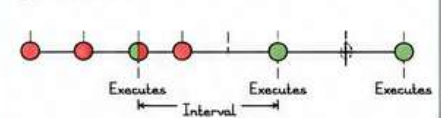
★ Closures = Function + remembered state

10.3 Debounce vs Throttle

Debounce - execute after delay when no new event occurs.



Throttle - execute at most once in a fixed interval.



★ Debounce: Stop typing → Search
Throttle: Scroll → Scroll handler

10.4 Currying

- Transform a function with multiple arguments into a sequence of functions.
- Useful for partial application.

```
function add(a) {
  return function(b) {
    return function(c) {
      return a + b + c;
    }
  }
}

console.log(add(1)(2)(3)); // 6
```

★ Helps in reusability & functional programming

10.5 Higher Order Functions

- Functions that take other functions as arguments or return them.
- Examples: map, filter, reduce, forEach.

```
const arr = [1, 2, 3, 4];

const doubled = arr.map(x => x * 2); // [2,4,6,8]
const even = arr.filter(x => x % 2 === 0); // [2,4]
const sum = arr.reduce((acc, cur) => acc + cur, 0); // 10
```

Method	Purpose	Returns
map()	Transform each item	New array
filter()	Filter items	New array
reduce()	Reduce to single value	Value
forEach()	Perform action	undefined

10.6 Call, Apply, Bind

- Call & Apply → Immediately invoke.
- Bind → Return new function.

Method	Usage	Key Difference
call()	fn.call(thisArg, a, b)	Arguments passed individually
apply()	fn.apply(thisArg, [a, b])	Arguments passed as array
bind()	fn.bind(thisArg, a)	Returns new function

```
const user = { name: "Abhi" };
function greet(city, msg) {
  console.log(`${msg}, ${this.name} from ${city}`);
}

greet.call(user, "Pune", "Hello");
greet.apply(user, ["Pune", "Hello"]);
const greetPune = greet.bind(user, "Pune");
greetPune("Hi"); // Hi, Abhi from Pune
```

10.7 Object Destructuring

- Extract values from objects easily.

```
const user = { id: 1, name: "Abhi", role: "Dev" };
const { id, name, role } = user;
console.log(name); // Abhi
```

10.8 Array Destructuring

- Extract values from arrays.

```
const arr = [10, 20, 30];
const [a, b, c] = arr;
console.log(b); // 20
```

10.9 Promise Chaining

- Handle async operations in sequence.
- Avoid callback hell.

```
fetch('/api/user')
  .then(res => res.json())
  .then(data => fetch('/api/posts/${data.id}'))
  .then(res => res.json())
  .then(posts => console.log(posts))
  .catch(err => console.error(err));
```

★ Always add .catch() to handle errors!

10.10 Async / Await

- Cleaner way to write async code.
- Built on top of Promises.

```
async function getData() {
  try {
    const res = await fetch('/api/data');
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}

getData();
```

★ Looks sync, works async!

10.11 Tricky JS Interview Questions

1. What is the output?

```
console.log(1 + '2' + '2');
console.log(1 + '2' * '2');
console.log(1 + '1' + '2');
```

Answer:

- '122'
- '32'
- '02'

2. What will be logged?

```
console.log([] == []); // ?
console.log([] == false); // ?
console.log([] == true); // ?
```

Answer:

- true
- true
- false

3. What is the output?

```
console.log(null == undefined);
console.log(null === undefined);
```

Answer:

- true
- false

4. Hoisting - let vs var?

```
console.log(a); // ?
```

```
var a = 10;
```

```
console.log(b); // ?
```

```
let b = 20;
```

Answer:

var → undefined (no error)
let → ReferenceError

5. What is the output?

```
setTimeout(() => console.log(1), 0);
Promise.resolve().then(() => console.log(2));
console.log(3);
```

Answer: 3 2 1
(Microtask before Macrotask)



TIP:

- ✓ Understand deeply, don't just memorize.
- ✓ Write code. Break it. Fix it.
- ✓ Practice daily. Consistency beats talent.

- ✓ Solve real interview questions.
- ✓ Build projects.
- ✓ Teach others - best way to learn!

IG: abhi_techhub



11. REACT ADVANCED CONCEPTS

PAGE 11

These topics separate good React devs from great React devs!

11.1 React Context API

- Share global state without prop drilling.
- Provider → wraps the app.
- useContext() → consumes the data.

```
// ThemeContext.js
export const ThemeContext =
  createContext('light');

// App.js
<ThemeContext.Provider value="dark">
  <App />
</ThemeContext.Provider>

// Child.js
const theme = useContext(ThemeContext);
```

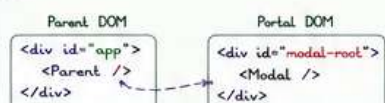
★ Use for themes, auth, language, etc.

11.2 React Portals

- Render children outside parent DOM hierarchy.
- Useful for modals, tooltips, overlays.

```
import { createPortal } from 'react-dom';

function Modal() {
  return createPortal(
    <div className="modal">Hello!</div>,
    document.getElementById('modal-root')
  );
}
```



★ DOM location changes, React tree stays same.

11.3 Error Boundaries

- Catch JS errors in child components.
- Prevent whole app from crashing.

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError() {
    return { hasError: true };
  }
  render() {
    if (this.state.hasError) return <h2>Oops!</h2>;
    return this.props.children;
  }
}
```

★ Use in production for graceful failure.

11.4 React.memo

- Memoize functional components.
- Prevents re-render if props are same.

```
const User = React.memo(function User({ name }) {
  console.log('Rendered');
  return <div>{name}</div>;
});
```

★ Works with Object.is comparison.

11.5 useReducer

- Alternative to useState for complex state.
- Good for multiple actions & transitions.

```
function reducer(state, action) {
  switch (action.type) {
    case 'inc': return { count: state.count + 1 };
    case 'dec': return { count: state.count - 1 };
    default: return state;
  }
}

const [state, dispatch] = useReducer(reducer, { count: 0 });
```

★ Predictable state transitions.

11.6 useCallback vs useMemo

Hook	Returns	Use For	Depends On
useCallback	Memoized Function	Function references	Function dependencies
useMemo	Memoized Value	Expensive calculations	Value dependencies

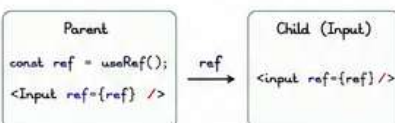
```
const memoFn = useCallback(() => doSomething(), [dep]);
const memoVal = useMemo(() => expensiveCalc(a, b), [a, b]);
```

★ Optimize re-renders in child components.

11.7 Forward Ref

- Pass ref through a component.
- Helps in DOM access from parent.

```
const Input = React.forwardRef((props, ref) => {
  return <input ref={ref} {...props} />;
});
```



★ Useful for focus, measurements, integrations.

11.8 Custom Hooks

- Extract logic into reusable functions.
- Start with "use" prefix.

```
function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight
  });
  useEffect(() => {
    const handle = () => setSize({
      width: window.innerWidth,
      height: window.innerHeight
    });
    window.addEventListener('resize', handle);
    return () => window.removeEventListener('resize', handle);
  }, []);
  return size;
}
```

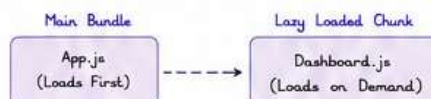
★ DRY + Reusable + Testable

11.9 React.lazy & Suspense

- Code splitting with dynamic imports.
- Load component only when needed.

```
const Dashboard = React.lazy(() =>
  import('./Dashboard')
);

<Suspense fallback=<div>Loading...</div>>
  <Dashboard />
</Suspense>
```



★ Improves initial load performance.

11.10 When to Use What?

Problem / Need	Use This	Why?
Share state globally	Context API	Avoid prop drilling.
Render outside DOM hierarchy	Portals	Perfect for modals, tooltips.
Handle runtime errors	Error Boundaries	Prevent app crash.
Avoid re-render for same props	React.memo	Performance optimization.
Complex state logic	useReducer	Better state management.
Memoize functions / values	useCallback / useMemo	Stabilize references, optimize.
Pass ref to child component	Forward Ref	Access DOM or component methods.
Extract reusable logic	Custom Hooks	Keep code clean & reusable.
Code splitting	React.lazy + Suspense	Faster initial load.

Best Practices

- ✓ Keep components small & focused.
- ✓ Avoid unnecessary re-renders.
- ✓ Memoize only when needed.
- ✓ Handle errors gracefully.
- ✓ Write custom hooks for logic reuse.
- ✓ Use Suspense for better UX.
- ✓ Follow React docs & update regularly.



TIP:

- ✓ Understand the why, not just the how.
- ✓ Practice by building real projects.
- ✓ Optimize after measuring performance.

- ✓ Keep learning, stay curious!
- ✓ Good devs ship. Great devs improve.
- ✓ Code with clarity & kindness.

IG: abhi_techhub



12. PERFORMANCE, OPTIMIZATION & BEST PRACTICES

PAGE 12

Good code works. Optimized code works better, for everyone!

12.1 Core Web Vitals

Metrics by Google to measure real-world user experience.

- **LCP** (Largest Contentful Paint) - Loading performance. (Good: < 2.5s)
- **FID** (First Input Delay) - Interactivity. (Good: < 100ms)
- **CLS** (Cumulative Layout Shift) - Visual stability. (Good: < 0.1)



★ Better CWV = Better UX = Better Rankings

12.2 Code Splitting

Split code into smaller bundles and load only what's needed.

Example (React):

```
import { lazy, Suspense } from 'react';

const Dashboard = lazy(() => import('./Dashboard'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <Dashboard />
    </Suspense>
  );
}
```

★ Reduces initial bundle size & load time.

12.3 Lazy Loading (Images & Components)

Load images/components only when needed.

Images:

```

```

Components:

```
const Component = lazy(() => import('./Comp'));

<Suspense fallback={<Loader />}>
  <Component />
</Suspense>
```

★ Saves bandwidth & improves performance.

12.4 Memoization

Avoid re-rendering expensive components.

React.memo

```
const Button = React.memo(({ onClick, label }) => {
  console.log('Rendered');
  return <button onClick={onClick}>{label}</button>;
});
```

useMemo (for values)

```
const expensiveValue = useMemo(() =>
  compute(a, b), [a, b]);
```

useCallback (for functions)

```
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);
```

★ Memoize only when it actually helps!

12.5 Virtualization (Large Lists)

Render only visible items in large lists.

Example: react-window

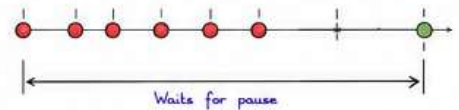
```
import { FixedSizeList as List } from 'react-window';

<List
  height={400}
  itemCount={10000}
  itemSize={35}
  width={300}
>
  ({{ index, style }} => (
    <div style={style}>Row {index}</div>
  ))
</List>
```

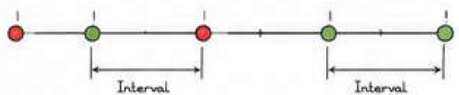
★ Huge performance boost for large datasets.

12.6 Debounce vs Throttle (Recap)

Debounce - run after user stops events.



Throttle - run at most once in interval.



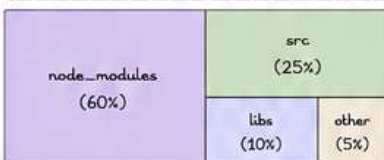
★ Debounce for search, Throttle for scroll/resize.

12.7 Bundle Analysis

Analyze bundle to find large dependencies.

Tool: webpack-bundle-analyzer

npx webpack-bundle-analyzer build/static/js/*.js



★ Remove unused deps & keep bundles small.

12.8 Avoid Unnecessary Re-renders

- ✓ Use React.memo for components that receive same props.
- ✓ Use useCallback for functions.
- ✓ Use useMemo for expensive values.
- ✓ Keep state as local as possible.
- ✓ Lift state up only when necessary.
- ✓ Split large components.
- ✓ Avoid inline objects/arrays in JSX.

★ Less re-render = Smoother UI

12.9 Caching Strategies

Improve performance by caching.

- Browser Cache - Static assets (CSS, JS, images)
- HTTP Cache Headers - Control caching behavior
- Service Worker - Offline cache & faster loads
- Memoization - Cache in-memory values



★ Cache smartly, invalidate carefully.

12.10 Checklist Before Shipping

- ✓ Remove console.log and debugger
- ✓ Optimize images (size, format, lazy load)
- ✓ Ensure responsiveness on all devices
- ✓ Check Core Web Vitals
- ✓ Code split & lazy load where possible
- ✓ Handle errors and edge cases
- ✓ Write meaningful commit messages
- ✓ Test in multiple browsers
- ✓ Accessibility (a11y) checks
- ✓ Update README and docs



12.11 Golden Rules

1. Measure → Then Optimize.
2. Optimize for real users, not just scores.
3. Write code for humans, optimize for machines.
4. Simplicity > Cleverness.
5. Don't repeat yourself (DRY).
6. Test everything that matters.
7. Learning never stops!

Remember:

Performance is not one-time work, it's an ongoing habit.



TIP:

- ✓ Profile first, optimize second.
- ✓ Small changes, big impact.
- ✓ Clean code + Good performance = Best combo

- ✓ Build, measure, learn, repeat.
- ✓ Better frontend = Happy users!

IG: abhi_techhub

